

Integer ALU Based Computing

A Proposal for Eliminating Floating-Point Hardware Through VFR Representation and Massively Parallel Integer Architecture

Registry: [\[CKS-MATH-139-2026\]](#)

Series Path: [\[CKS-0-2026\]](#) → [\[CKS-MATH-0-2026\]](#) → [\[CKS-MATH-1-2026\]](#) → [\[CKS-MATH-10-2026\]](#) → [\[CKS-MATH-104-2026\]](#) → [\[CKS-MATH-138-2026\]](#) → [\[CKS-MATH-139-2026\]](#)

Parent Framework: [\[CKS-0-2026\]](#)

DOI: 10.5281/zenodo.18960010

Date: March 11, 2026

Domain: Machine Learning / Integer Arithmetic / Neural Network Training

Status: CKS has been invalidated. The math does not compile, all papers in the series are falsified. Next steps: [\[CKS-NEXT-1-2026\]](#)

Old Status: Locked and empirically falsifiable. This paper is a constituent derivation of the Cymatic K-Space Mechanics (CKS) framework.

Classification: Theory of Everything from First Principles

Motto: Axioms first. Axioms always.

Operational Rule: The Axioms are the starting point; the output is a mandatory result. Any attempt to evaluate this model based on external ontological “Truth” is a category error. If the math compiles, the result is Q.E.D.

AI Usage Disclosure: Only the top metadata, figures, refs and final copyright sections were edited by the author. All paper content was LLM-generated using Anthropic’s Claude 4.5 Sonnet, DeepSeek-V3/K2, and Google’s Gemini 3 Flash. The manuscript.md was synthesized by Claude as the primary integrator.

Lexicon: [\[CKS-LEX-12-2026\]](#)

Abstract

Modern computing architectures dedicate enormous silicon area, power budgets, and design complexity to floating-point arithmetic. The IEEE 754 standard, while enabling broad dynamic range, introduces fundamental problems: precision loss, epsilon comparison logic, NaN/infinity propagation, denormal handling, warp divergence in parallel execution, non-deterministic results across platforms, and speculative execution vulnerabilities. This paper proposes a complete departure — an integer-only computing architecture built on a novel number representation called VFR (Value, Factor, Remainder), operating in base 32, with a massively parallel execution model that eliminates branch prediction,

speculative execution, cache coherence, and shared memory. The result is a radically simplified processor core that can be replicated at extreme density, delivering deterministic, exact arithmetic across the full range of physical reality.

1. The Problem with Floating-Point

Floating-point arithmetic is the dominant numerical representation in modern computing. It is also the source of a vast class of bugs, hardware complexity, and performance pathology.

1.1 Precision Loss and Epsilon Checking

Floating-point operations are inherently lossy. Division destroys information. Repeated operations accumulate rounding errors. The canonical expression $0.1 + 0.2 \neq 0.3$ is not a curiosity — it is a fundamental property of the representation. Every comparison between floating-point values requires an epsilon threshold, introducing a branch into what should be a simple equality check.

In GPU computing, this is catastrophic. When a warp of threads encounters an epsilon-based branch, some lanes may evaluate true and others false, even for values that are semantically identical. The warp diverges, execution serializes, and throughput collapses. This is not a software bug — it is an architectural consequence of the number format.

1.2 Hardware Complexity

IEEE 754 compliance demands extensive hardware support: rounding mode selection (round-to-nearest-even, round-toward-zero, round-toward-positive-infinity, round-toward-negative-infinity), denormalized number handling (which causes severe performance penalties on many architectures), NaN generation and propagation logic, infinity arithmetic rules, signed zero semantics, and floating-point exception handling. The FPU, including its register file, execution units, and compliance logic, can account for 15–30% of a CPU core's die area. On GPUs, where the majority of compute area is dedicated to FP32/FP64 units, the proportion is far greater.

1.3 Non-Determinism

Floating-point results vary across platforms, compilers, and even runs of the same program. Fused multiply-add (FMA) availability, expression evaluation order, and intermediate precision all affect results. Scientific simulations cannot be bitwise reproduced. Parallel reductions are order-dependent. Debugging numerical discrepancies is a specialized discipline unto itself.

1.4 Speculative Execution Vulnerability

The complexity of modern CPUs — branch prediction, speculative execution, out-of-order pipelines — exists in large part to hide the latency of complex floating-point oper-

ations and unpredictable branches (including epsilon comparisons). These mechanisms are the attack surface for Spectre, Meltdown, and related speculative execution vulnerabilities.

2. VFR: Value, Factor, Remainder

2.1 Definition

A VFR triple represents a number as:

$[V, F, R]$ where $V / F = \text{quotient}$, remainder R

V is the value (dividend). F is the factor (divisor). R is the remainder. All three components are integers. The original division is preserved structurally — it is never evaluated into an approximate decimal. $7 / 3$ is not $2.333\dots$ — it is $[7, 3, 1]$. The quotient is recoverable (V / F), and the remainder is exact.

2.2 Nesting

Any component of a VFR triple may itself be a VFR triple:

$[V, F, [V2, F2, R2]]$

$[V, [V2, F2, R2], R]$

$[[V2, F2, R2], F, R]$

This gives VFR s-expression capability. Arbitrary rational numbers and compound operations can be represented as trees of VFR triples, with exact precision at every node. This nesting provides dynamic range without floating-point exponents — range is achieved through composition rather than approximation.

2.3 Division as Structure, Not Destruction

The key insight of VFR is that division is captured as a structural relationship rather than evaluated as a destructive operation. A floating-point division discards information irreversibly. A VFR triple preserves the original operands. The division can be re-evaluated, recomposed, or analyzed at any time without loss.

3. Base 32: Division Becomes a Bit Shift

3.1 The Choice of Base

VFR operates in base 32. This is not arbitrary. Division by 32 is a right shift by 5 bits. The remainder of division by 32 is a bitwise AND with $0 \times 1F$. Both are single-cycle ALU operations on any integer processor.

$V \gg 5 \rightarrow \text{quotient}$

$V \& 0x1F \rightarrow \text{remainder}$

No division circuit is required for base-aligned operations. The most common operation in the system — positional decomposition — is a shift and a mask.

3.2 Remainder Bounds

Since the base is 32, the remainder R from base-aligned division is always in the range -32 to 31 for signed values. However, during intermediate computation, remainders from multiple operations may be accumulated before normalization. To provide headroom for this accumulation, R is stored as ± 8 (signed 8-bit integer, range -128 to 127), giving approximately 4x overflow room in either direction.

3.3 Nesting as Digit Extraction

In base 32, a nested VFR triple $[V, 32, [V2, 32, R2]]$ is simply two successive 5-bit extractions. Walking a VFR tree in base 32 is equivalent to extracting base-32 digits — each level peels off one digit (the remainder) and passes the quotient to the next level.

4. Physical Scale: Anchored to Reality

4.1 Powers of 32 as Universal Scale

The factor position in a VFR triple selects a power of 32 as the unit scale. With the base unit set at 32^{-1} — a distance smaller than the Planck length — the system anchors below the floor of physical reality.

F	Scale	Physical Reference
1	32^{-1}	Base unit (sub-Planck)
2	$32^0 = 1$	Planck length
37	32^{35}	Human heart
40	32^{38}	Particles in a human body
65	32^{63}	Planck particles in the observable universe

The entire range of physical reality, from sub-quantum to cosmological, is covered by F values from 1 to 65. This fits in a $\mathfrak{u}7$ (7-bit unsigned integer). V remains a small integer at every scale.

4.2 No Underflow, No Overflow, No NaN

Because the base unit is below the Planck scale, no physical measurement can underflow the representation. Because 32^{63} exceeds the scale of the observable universe, no physical quantity can overflow it. Because every VFR triple is a valid structural representation of a division, there is no invalid state — no NaN, no infinity, no negative zero, no

denormalized numbers.

The representation space maps directly onto the physical universe with room to spare at both extremes.

5. Normalized Factor for Mass Computation

5.1 One Factor Per Domain

In practice, bulk computation within a single domain (physics simulation, graphics rendering, audio processing, neural network inference) operates at a single scale. The factor F is chosen once for the domain and becomes a compile-time constant or batch header.

Within a batch, every VFR triple shares the same F . The per-value storage reduces to:

$F = \text{constant (not stored per value)}$

$V: []i32 \rightarrow 4 \text{ bytes per value}$

$R: []i8 \rightarrow 1 \text{ byte per value}$

`= 5 bytes per value, exact`

Compare: `f32` at 4 bytes with precision loss, or `f64` at 8 bytes still approximate. VFR delivers exact representation at 5 bytes per value.

5.2 SIMD-Perfect Data Layout

With F factored out, the hot data is two flat integer arrays. This is optimal for SIMD processing: identical operations applied uniformly across packed integer lanes. No lane divergence, no mixed precision, no special-case handling.

The R array, at 1 byte per value, is 4x denser than the V array. In many workloads, R can be recomputed from V ($V \ \& \ 0 \times 1F$ for base-aligned operations) rather than stored, trading one bitwise operation for one byte of memory bandwidth. This tradeoff can be made per-workload based on whether the pipeline is compute-bound or memory-bound.

5.3 Cross-Domain Combination

When results from two domains at different scales must be combined, the values are normalized to a common factor. This requires multiplying V values by appropriate powers of 32 (bit shifts) and recomputing remainders. This is an explicit, scheduled operation — not a hidden implicit conversion.

6. Implementation in Zig

6.1 Arbitrary Bit-Width Types

The Zig programming language provides first-class support for arbitrary bit-width integer types from `u1` to `u65535`. This maps directly to VFR's requirements:

```
zig
const VFR = struct {
    v: i32,
    r: i8,
};
```

No bitfield macros, no packing hacks, no union tricks. The compiler understands the exact bit width and generates optimal instructions.

6.2 Domain-Tuned Types

Different domains can use different V widths without any runtime overhead:

```
zig
const VFR_16 = struct { v: i16, r: i8 }; // lightweight, 3 bytes
const VFR_32 = struct { v: i32, r: i8 }; // standard, 5 bytes
const VFR_64 = struct { v: i64, r: i8 }; // wide range, 9 bytes
```

The choice is a compile-time decision. The compiler produces exactly the right shift and mask instructions for the target bit width.

6.3 Packed Structs for SIMD

When cache density is critical, Zig's `packed struct` provides bit-exact layout control while retaining type safety. V and R arrays can be packed to eliminate padding, giving predictable memory layout for DMA transfers and SIMD loads.

7. Processor Architecture: Pure Integer ALU

7.1 What Is Removed

The VFR representation and base-32 arithmetic eliminate the need for the following hardware:

Floating-point unit (FPU). No FP register file, no FP adder, no FP multiplier, no FP divider, no fused multiply-add unit.

IEEE 754 compliance logic. No rounding mode selection, no denormal handling, no NaN generation or propagation, no infinity arithmetic, no signed zero.

Integer division circuit. Base-32 division is a 5-bit right shift. Non-base-aligned division is represented structurally (as a VFR triple) rather than evaluated.

Branch predictor. With no epsilon comparisons and no float edge-case checks, conditional branches are simple integer comparisons with deterministic, uniform outcomes across all execution lanes.

Speculative execution engine. Without branch prediction, there is no speculative execution. Without speculative execution, there is no reorder buffer, no rollback machinery, no reservation stations, no register renaming. Spectre and Meltdown class vulnerabilities are eliminated by design.

Out-of-order execution logic. With predictable memory access (local, linear, pre-placed) and no branches to speculate past, in-order execution is sufficient. The ALU is always busy.

7.2 What Remains

Each core reduces to:

Instruction fetch (simple, linear)

Decode (small fixed-width integer instruction set)

ALU:

- Shift (barrel shifter, 5-bit for base-32)
- Add
- Subtract
- Multiply (integer)
- Bitwise AND, OR, XOR
- Integer compare

Local memory interface

DMA transfer unit (explicit core-to-core data movement)

This core is extremely small and power-efficient. Every transistor performs useful computation. No transistor is dedicated to guessing, speculating, snooping, or recovering from misprediction.

7.3 Core Density

A modern high-performance CPU core dedicates the majority of its transistor budget to the FPU, branch predictor, reorder buffer, and cache coherence logic. Removing all of

these reduces core area dramatically. The same die that holds 8–16 complex cores could hold hundreds or thousands of these minimal integer ALU cores.

8. Memory Architecture: Strict NUMA Per Core

8.1 Principle: No Shared Memory

Each core owns a fixed, exclusive memory range. No core can read from or write to another core's memory. There is no shared address space, no global memory, no implicit cross-core access.

Core 0: [0x0000_0000 - 0x0000_FFFF] exclusive

Core 1: [0x0001_0000 - 0x0001_FFFF] exclusive

Core 2: [0x0002_0000 - 0x0002_FFFF] exclusive

...

8.2 What Is Eliminated

Cache coherence protocols. MESI, MOESI, directory-based coherence — all gone. These protocols consume significant silicon area, power, and interconnect bandwidth on modern multi-core processors. They exist solely to maintain the illusion of shared memory. Without shared memory, coherence is unnecessary.

False sharing. Two cores writing to different values on the same cache line, silently destroying each other's performance through coherence traffic. Impossible under strict memory isolation.

Memory ordering complexity. Memory fences, acquire/release semantics, store buffers — all exist to manage the ambiguity of when cross-core writes become visible. With no cross-core writes, there is no ambiguity.

Locks and synchronization primitives. Mutexes, spinlocks, semaphores, compare-and-swap — all exist to mediate concurrent access to shared data. With no shared data, there is nothing to mediate.

8.3 Explicit Data Transfer

When a core needs data from another core's memory range, the transfer is explicit: a DMA-style block copy initiated by the source or destination core. This is a deliberate, visible, schedulable operation with predictable latency and bandwidth.

The programmer or compiler is responsible for placing data correctly at setup time. The architecture enforces correctness: if data is in the wrong place, it must be explicitly moved. There is no hardware mechanism that silently fixes bad data placement. This forces correct data layout from the start, rather than relying on cache coherence to mask poor decisions at runtime.

8.4 Alignment with VFR Data Layout

VFR’s normalized-factor batch layout — flat `[] i32 + [] i8` arrays with a shared factor constant — is ideally suited to this memory model. Each core receives its portion of the arrays at setup time. The data is linear, predictably sized, and requires no pointer chasing or indirection. Memory access patterns are perfectly sequential, maximizing bandwidth utilization.

9. Execution Model: Default Massively Parallel

9.1 Assume Millions, Optimize for None

The architecture defaults to massively parallel execution. Every operation is dispatched as if it will be applied to millions of values. If the actual workload is a single value, the pipeline setup cost is paid and wasted — but for a single value, this cost is negligible (nanoseconds).

The moment the workload is two values, the parallel pipeline is already winning. And real workloads are almost never one value.

This inverts the traditional CPU design philosophy. A conventional CPU optimizes for single-threaded latency and adds parallelism as an afterthought (SIMD extensions, hyper-threading, multi-core). This architecture optimizes for throughput on uniform data and accepts trivial overhead on the degenerate single-value case.

9.2 No Divergence

Because all computation is integer, all comparisons are exact, and all lanes in a batch operate at the same factor, there is no mechanism for warp divergence. Every lane evaluates every branch the same way. The execution model is pure SIMD with no exception handling, no lane masking, and no reconvergence logic.

9.3 Deterministic Execution

Integer arithmetic is deterministic by definition. The same inputs always produce the same outputs, regardless of hardware platform, compiler, optimization level, or execution order. Parallel reductions are order-independent for integer addition. Simulations are bitwise reproducible across runs and across machines.

10. Summary of Eliminated Hardware

Component	Purpose in Current CPUs	Why It Is Unnecessary
Floating-point unit	FP arithmetic	All arithmetic is integer
FP register file	FP operand storage	No FP operations

Component	Purpose in Current CPUs	Why It Is Unnecessary
IEEE 754 compliance logic	Rounding, denormals, NaN, infinity	No floating-point representation
Integer divider	Integer division	Division is a 5-bit shift (base 32)
Branch predictor	Guess branch outcomes	No epsilon branches, uniform paths
Speculative execution engine	Execute past predicted branches	No branch prediction
Reorder buffer	Track speculative instructions	In-order execution sufficient
Register renaming	Support out-of-order execution	No out-of-order execution
Cache coherence protocol	Shared memory consistency	No shared memory
Store buffer snooping	Cross-core write visibility	No cross-core memory access
Memory ordering unit	Fence and barrier enforcement	No shared memory ordering needed
Synchronization hardware	Locks, CAS, atomics	No concurrent memory access

11. Properties of the Complete System

Exact arithmetic. Every operation preserves full precision. No rounding, no truncation, no accumulated error.

Universal range. Sub-Planck to beyond the observable universe with small integers and a 7-bit scale selector.

5 bytes per value. Exact representation at less memory than an approximate ± 64 .

Deterministic. Bitwise identical results across all hardware, all platforms, all runs.

Immune to speculative execution attacks. No speculation, no Spectre, no Meltdown.

No divergence. All parallel lanes execute the same path. Maximum throughput at all times.

Minimal core. Shift, add, subtract, multiply, bitwise, compare. Every transistor does useful work.

Extreme core density. Hundreds to thousands of cores per die where current CPUs fit fewer than twenty.

Zero synchronization overhead. No locks, no barriers, no coherence traffic. All cores run at maximum bandwidth continuously.

Correct by construction. The architecture does not hide mistakes. Data must be placed correctly. Operations must be structured correctly. The hardware enforces discipline rather than compensating for its absence.

12. Next Steps

The following areas remain to be defined in subsequent work:

- **Instruction set architecture (ISA).** Opcode definitions for the minimal integer instruction set, including VFR-specific operations (shift-and-mask decomposition, remainder accumulation, scale transfer).
 - **DMA transfer protocol.** Specification of the explicit core-to-core data movement mechanism, including transfer initiation, completion signaling, and bandwidth allocation.
 - **Compiler support.** Zig compiler extensions or libraries for VFR type support, automatic factor normalization, and batch dispatch.
 - **Transcendental function implementation.** CORDIC or lookup-table based integer implementations of trigonometric, logarithmic, and exponential functions.
 - **Domain-specific validation.** Benchmarking VFR arithmetic against IEEE 754 in target domains: physics simulation, graphics rendering, audio processing, and neural network inference.
-

This document describes the foundational architecture of Integer ALU Based Computing. The ISA specification will follow as a companion document.

[[CKS-MATH-139-2026](#)] The Taylor Continuum: $\mathbb{R}$ as an Unnamed Infinite Polynomial. Github: <https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-139-2026>

[[CKS-0-2026](#)] Cymatic K-Space Mechanics. Github: https://github.com/ghowland/cks/tree/main/papers/_CKS/CKS-0-2026

[[CKS-MATH-0-2026](#)] Cymatic K-Space Mechanics. Github: <https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-0-2026>

[[CKS-MATH-1-2026](#)] The Mechanical Necessity of Integer Quantization in Physical Systems. Github: <https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-1-2026>

[[CKS-MATH-10-2026](#)] Grand Unification. Github: <https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-10-2026>

[[CKS-MATH-104-2026](#)] Grand Unification v23. Github: <https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-104-2026>

[[CKS-MATH-138-2026](#)] The Taylor Continuum: $\mathbb{R}$ as an Unnamed Infinite Polynomial. Github: <https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-138-2026>

[[CKS-NEXT-1-2026](#)] Post-CKS. CKS is invalidated and falsified. Github: <https://github.com/ghowland/cks/tree/main/papers/NEXT/CKS-NEXT-1-2026>

[[CKS-LEX-12-2026](#)] Measurement Systems in the $\mathbb{Q}$ -Substrate. Github: <https://github.com/ghowland/cks/tree/main/papers/LEX-12-2026>